Lab 7 Short Report

Variant 4: English Words to Number

Group Number: 34
Students: Engin S. Demirkan Franziska Axmann

May 22, 2026

1 Problem Description and Logic Flow

The objective of this exercise is to build a Prolog program that converts the English word representation of a number (up to 1,000) into its corresponding numerical integer, complying with the constraint $N \leq 1000$. For example, transforming an input like `"three hundred and ninety four"` into `394`.

The logic flow of the solution is structured as a single-direction pipeline:

1. **Input Reception:** The user provides a double-quoted string representing a number in English.
2. **Pre-processing (Lexical Analysis):** The string is converted to lowercase using `string_lower/2` to achieve full case-insensitivity. It is then tokenized by spaces via `split_string/4`, and any empty tokens resulting from multiple consecutive spaces are filtered out using `include/3`. Finally, string pieces are mapped into Prolog atoms via `atom_string/2`.
3. **Parsing (Syntactic Analysis):** The resulting clean list of atoms is parsed using a Definite Clause Grammar (DCG). The grammar mirrors the natural hierarchy of English numbers (breaking down thousands, hundreds, tens, and ones).
4. **Evaluation and Arithmetic Compilation:** As the DCG successfully navigates and matches the grammatical structures, it recursively executes semantic actions to dynamically compute the total numeric value.

2 Main Components of the Code

The source code is organized into four main functional blocks:

- **Fact Dictionaries (`ones/2`, `teens/2`, `tens_word/2`):** These form the atomic knowledge base of the program, mapping individual linguistic tokens (e.g., `zero`, `fourteen`, `sixty`) directly to their integer equivalents (`0`, `14`, `60`).
- **String Tokeniser (`string_to_atom_list/2`):** This predicate isolates and sanitizes the user's string input, transforming it into an order-preserved list of atoms ready for grammatical analysis.
- **DCG Grammar (`number//1`, `hundreds//1`, `below_hundred//1`):** The core engine. It defines the sentence structures allowed under British English conventions, strictly enforcing the presence of the word `and` between hundreds and subsequent remainders.
- **Public Predicate (`to_num/2`):** The user-facing API entry-point which coordinates the string transformation and invokes the DCG parser via `phrase/2`.

3 Challenges and Edge Cases Resolved

Throughout development, several crucial edge cases were uncovered and robustly handled to guarantee an optimal solution:

- **Case Sensitivity:** Initially, the parser failed if words were capitalized (e.g., `"One Hundred"`). Incorporating `string_lower/2` during the pre-processing pipeline rendered the program immune to case variation.
- **Handling the "Zero" Input:** Standard numeric grammar structures often assume values begin at 1. Explicitly introducing `ones(zero, 0)` ensured that `to_num("zero", N)` evaluates correctly to `0` instead of returning a failure state.
- **Whitespace and Formatting Irregularities:** Accidental double spacing (e.g., `"forty five"`) could disrupt naive tokenizers. Utilizing the higher-order predicate `include/3` to discard empty string elements successfully insulated the grammar rules from syntax errors.